

UNIT II

DBMS LANGUAGES

DDL Data Definition Language.

DDL includes the SQL commands that can be used to **define and modify the structure (schema)** of database objects like tables, views, and indexes.

- **CREATE, ALTER, DROP, TRUNCATE**

```
CREATE TABLE Student (  
    Roll_No INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

DML Data Manipulation Language.

DML includes the SQL commands that can be used to **manage data stored in the database**. DML is used to **insert, update, delete, and retrieve data** stored in the database.

- **SELECT, INSERT, UPDATE, DELETE**

```
INSERT INTO Student VALUES (101, 'Rahul', 20);
```

DCL Data Control Language

DCL includes the SQL commands that can be used to **control access permissions and security in the database**. This includes granting and/or revoking permissions and managing privileges.

- GRANT and REVOKE.

```
GRANT SELECT ON Student TO user1;
```

TCL Transaction Control Language

TCL statements Transaction Control Language (**TCL**) in **SQL** are used to manage database transactions and ensure data integrity in relational databases.

- COMMIT, ROLLBACK, SAVEPOINT

```
COMMIT;
```

Basis	DDL (Data Definition Language)	DML (Data Manipulation Language)
Purpose	Defines and modifies the structure of the database	Manipulates the data stored in the database
Focus	Schema (tables, views, indexes)	Records (rows/tuples)
Commands	CREATE, ALTER, DROP, TRUNCATE	SELECT, INSERT, UPDATE, DELETE
Effect on Data	Changes structure, not actual data directly	Directly affects the data
Auto Commit	Usually auto-committed (cannot be rolled back easily)	Can be rolled back (uses COMMIT/ROLLBACK)
Usage	Used during database design and modification	Used during data handling and operations
Example	CREATE TABLE Student(id INT);	INSERT INTO Student VALUES(1);
Control	Controlled by DBA or designers	Used by end users, analysts, programmers

Queries on Tables

Create

```
CREATE TABLE Student (  
    Roll_No INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT,  
    Course VARCHAR(50)  
);
```

Insert

```
INSERT INTO Student (Roll_No, Name, Age, Course)  
VALUES (101, 'Rahul', 20, 'Computer Science');
```

Update

```
UPDATE Student  
SET Age = 21  
WHERE Roll_No = 101;
```

Delete

```
DELETE FROM Student  
WHERE Roll_No = 101;
```

Alter

```
ALTER TABLE Student  
ADD Email VARCHAR(100);
```

Drop

```
DROP TABLE Student;
```

Select

```
SELECT * FROM Student;
```

Joins

A JOIN in SQL is an operation that combines rows from two or more tables based on a related column, allowing related data stored in separate tables to be retrieved together in a single result.

StudentID	Name	DeptID		DeptID	DeptName
1	Asha	10		10	IT
2	Ravi	20		20	Computer
3	Neha	30		40	Mechanical

Inner Join

An inner join is an SQL operation that returns only the rows that have matching values in both tables based on a specified condition. Non-matching rows are excluded from the result.

```

SELECT Students.StudentID, Students.Name, Departments.DeptName
FROM Students
INNER JOIN Departments
ON Students.DeptID = Departments.DeptID;

```

StudentID	Name	DeptName
1	Asha	IT
2	Ravi	Computer

Left Outer Join

A left join is an SQL operation that returns all rows from the left table and the matching rows from the right table. If no match exists, NULL values are returned for columns of the right table.

```

SELECT Students.StudentID, Students.Name, Departments.DeptName
FROM Students
LEFT JOIN Departments
ON Students.DeptID = Departments.DeptID;

```

StudentID	Name	DeptName
1	Asha	IT
2	Ravi	Computer
3	Neha	NULL

Right Outer Join

A right join is an SQL operation that returns all rows from the right table and the matching rows from the left table. If no match exists, NULL values are returned for columns of the left table.

```

SELECT Students.StudentID, Students.Name, Departments.DeptName
FROM Students
RIGHT JOIN Departments
ON Students.DeptID = Departments.DeptID;

```

StudentID	Name	DeptName
1	Asha	IT
2	Ravi	Computer
NULL	NULL	Mechanical

Full Outer Join

A full outer join is an SQL operation that returns all rows from both tables, including matching and non-matching rows, with NULL values filled in where no match is found.

```

SELECT Students.StudentID, Students.Name, Departments.DeptName
FROM Students
FULL OUTER JOIN Departments
ON Students.DeptID = Departments.DeptID;

```

StudentID	Name	DeptName
1	Asha	IT
2	Ravi	Computer
3	Neha	NULL

StudentID	Name	DeptName
NULL	NULL	Mechanical

Views

- A **view** is a virtual table in SQL that displays data stored in one or more tables.
- A view does not store data physically.
- Views are used to simplify complex queries and provide restricted access to data.
Example: If a table **Employee(EmpID, Name, Salary, Dept)** exists, a view can show only selected columns such as Name and Department.

Advantages of views:

1. **Data Security:** Views help restrict access to sensitive data by allowing users to see only selected rows or columns of a table.
2. **Simplifies Complex Queries:** Views can store complex SQL queries, such as joins and conditions, so they can be reused easily without rewriting them.
3. **Data Abstraction:** Views hide the actual table structure and let users interact with a simplified version of the data.
4. **Data Consistency:** The same view can be used by multiple users, ensuring consistent results.
5. **Logical Data Independence:** Changes in the underlying tables may not affect users if the view definition remains unchanged.

CREATE A VIEW:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

```
CREATE VIEW Employee_View AS
SELECT Name, Dept
FROM Employee;
```

UPDATE A VIEW:

```
UPDATE view_name
SET column_name = value
WHERE condition;
```

```
UPDATE Employee_View
SET Dept = 'HR'
WHERE Name = 'Rahul';
```

DELETE A VIEW:

```
DROP VIEW view_name;
```

Aggregate Functions

An aggregate function is an SQL function that performs a calculation on multiple rows of a table and returns a single scalar value. They are commonly used to summarize data in a table such as totals, averages, counts, or extreme values.

SQL offers five built-in aggregate functions :

- **AVG:** returns the average value of a column.
- **COUNT:** returns the number of rows in a column.
- **SUM:** returns the sum of values in a column.
- **MIN:** returns the minimum value in a column.
- **MAX:** returns the maximum value in a column.

Set operations

- Set operations in DBMS are used to combine the results of two or more SELECT queries.
- The main requirement is that all queries must have the same number of columns and compatible data types.

Student_IT Student_CS

ID	Name		ID	Name
1	Asha		3	Neha
2	Ravi		4	John
3	Neha		5	Maya

UNION: It combines the results of two SELECT queries and removes duplicate rows.

```
SELECT Name FROM Students_IT
UNION
SELECT Name FROM Students_CS;
```

Result:

Asha, Ravi, Neha, John, Maya

UNION ALL: It combines the results of two SELECT queries and keeps all duplicate rows.

```
SELECT Name FROM Students_IT
UNION
SELECT Name FROM Students_CS;
```

Result:

Asha, Ravi, Neha, Neha, John, Maya

INTERSECT: It returns only the common rows that appear in both SELECT queries.

MINUS (or EXCEPT): It returns rows from the first SELECT query that are not present in the second query.

```
SELECT Name FROM Students_IT
INTERSECT
SELECT Name FROM Students_CS;
```

Result:

Neha

```
SELECT Name FROM Students_IT
MINUS
SELECT Name FROM Students_CS;
```

Result:

Asha, Ravi

SQL Operators

SQL Arithmetic Operators

Operator	Description	Example
+	Addition	SELECT 5 + 3;
-	Subtraction	SELECT 10 - 4;
*	Multiplication	SELECT 6 * 2;
/	Division	SELECT 20 / 5;

Operator	Description	Example
%	Modulus	SELECT 10 % 3;

SQL Comparison Operators

Operator	Description	Example
=	Equal to	SELECT * FROM Students WHERE Marks = 80;
>	Greater than	SELECT * FROM Students WHERE Marks > 75;
<	Less than	SELECT * FROM Students WHERE Marks < 50;
>=	Greater than or equal to	SELECT * FROM Students WHERE Marks >= 60;
<=	Less than or equal to	SELECT * FROM Students WHERE Marks <= 40;
<>	Not equal to	SELECT * FROM Students WHERE Marks <> 90;

SQL Compound Operators

Operator	Description	Example
+=	Add equals	UPDATE Accounts SET Balance += 500 WHERE ID = 1;
-=	Subtract equals	UPDATE Accounts SET Balance -= 200 WHERE ID = 1;
*=	Multiply equals	UPDATE Products SET Price *= 2 WHERE ID = 1;
/=	Divide equals	UPDATE Products SET Price /= 2 WHERE ID = 1;
%=	Modulo equals	UPDATE Numbers SET Value %= 2 WHERE ID = 1;
&=	Bitwise AND equals	UPDATE Flags SET Value &= 1 WHERE ID = 1;
^=	Bitwise XOR equals	UPDATE Flags SET Value ^= 1 WHERE ID = 1;
=	Bitwise OR equals	UPDATE Flags SET Value

SQL Bitwise Operators

Operator	Description	Example
&	Bitwise AND	SELECT 5 & 3;
	Bitwise OR	SELECT 5 \ 3;
^	Bitwise XOR	SELECT 5 ^ 3;
~	Bitwise NOT	SELECT ~5;

SQL Logical Operators

Operator	Description	Example
ALL	TRUE if all values meet condition	SELECT * FROM Students WHERE Marks > ALL (SELECT Marks FROM Students WHERE DeptID = 10);
AND	TRUE if all conditions are TRUE	SELECT * FROM Students WHERE Marks > 50 AND Age < 20;
ANY	TRUE if any value meets condition	SELECT * FROM Students WHERE Marks > ANY (SELECT Marks FROM Students WHERE DeptID = 10);
BETWEEN	Within a range	SELECT * FROM Students WHERE Marks BETWEEN 50 AND 80;
EXISTS	TRUE if subquery returns rows	SELECT * FROM Students S WHERE EXISTS (SELECT 1 FROM Marks M WHERE S.ID = M.ID);
IN	Matches any value in list	SELECT * FROM Students WHERE DeptID IN (10, 20);
LIKE	Pattern matching	SELECT * FROM Students WHERE Name LIKE 'A%';
NOT	Negates condition	SELECT * FROM Students WHERE NOT Marks > 50;
OR	TRUE if any condition is TRUE	SELECT * FROM Students WHERE Marks > 80 OR Age < 18;

Operator	Description	Example
SOME	Same as ANY	SELECT * FROM Students WHERE Marks > SOME (SELECT Marks FROM Students WHERE DeptID = 10);

SQL String Operators

Operator	Description	Example
(Concatenation)	Combines two strings	SELECT 'Hello' 'World';
CONCAT()	Joins strings	SELECT CONCAT(Name, ' - ', DeptID) FROM Students;
LENGTH()	Returns length of string	SELECT LENGTH('Hello');
UPPER()	Converts to uppercase	SELECT UPPER(Name) FROM Students;
LOWER()	Converts to lowercase	SELECT LOWER(Name) FROM Students;

Nested Query

In nested queries, a query is written inside a query. The result of inner query is used in execution of outer query.

Types:

- **Independent Subquery:** An independent subquery is a nested query that can be executed on its own without relying on the outer query. The result of the subquery is first evaluated and then used by the outer query.

```
SELECT Name FROM Students WHERE Age > (SELECT AVG(Age) FROM Students);
```

Here, the subquery (SELECT AVG(Age) FROM Students) runs independently and gives a single value, which is then used by the outer query.

- **Correlated Subquery:** A correlated subquery is a nested query that depends on the outer query for its values. It cannot be executed independently because it runs once for each row processed by the outer query.

```
SELECT S1.Name
FROM Students S1
WHERE S1.Age >
      (SELECT AVG(S2.Age)
       FROM Students S2
       WHERE S2.DeptID = S1.DeptID);
```

Here, the inner query uses S1.DeptID from the outer query, so it runs repeatedly for each row of the outer query.

Clauses

ORDER BY Clause

The ORDER BY clause is used to sort the result of a query in ascending or descending order.

```
SELECT Name, Marks
FROM Students
ORDER BY Marks DESC;
```

GROUP BY Clause

The GROUP BY clause is used to group rows that have the same values in specified columns, usually with aggregate functions.

Example:

```
SELECT DeptID, AVG(Marks)
FROM Students
```

```
GROUP BY DeptID;
```

HAVING Clause

The **HAVING** clause is used to filter groups created by **GROUP BY** based on a condition.

```
SELECT DeptID, AVG(Marks)
FROM Students
GROUP BY DeptID
HAVING AVG(Marks) > 70;
```

WHERE	HAVING
Filters individual rows	Filters grouped results
Applied before GROUP BY	Applied after GROUP BY
Cannot use aggregate functions	Can use aggregate functions
SELECT DeptID, AVG(Marks) FROM Students WHERE Marks > 50 GROUP BY DeptID;	SELECT DeptID, AVG(Marks) FROM Students GROUP BY DeptID HAVING AVG(Marks) > 70;

PL/SQL

- PL/SQL is a procedural language.
- The code is written in small blocks in PL/SQL.
- A PL/SQL block has a name.
- A Function or a Procedure is an example of a named block.

A PL/SQL block consists of three sections : declaration, executable, and exception-handling sections.

In a block, the **executable section is mandatory** while the **declaration and exception-handling sections are optional**.

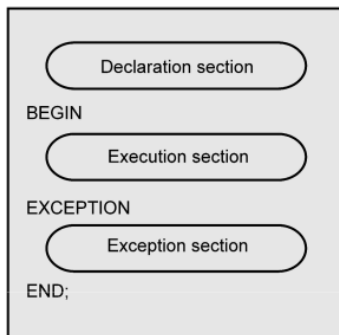


Fig. 3.3.1 Block structure of PL/SQL

1) Declaration Section:

This section is used to declare variables, constants, cursors, and data types required in the program. It starts with the keyword `DECLARE` (optional).

2) Executable Section:

This is the main part of the PL/SQL block where actual statements are written and executed. It starts with `BEGIN` and ends with `END`. At least one statement must be present.

3) Exception Handling Section:

This section handles errors that occur during execution. It starts with the keyword `EXCEPTION` and defines how to respond to different errors.

Stored Procedure

A stored procedure is a named, precompiled block of SQL (and PL/SQL) statements stored in the database that performs a specific task and can be executed whenever required. It improves performance, reusability, and simplifies complex operations

SYNTAX:

```
CREATE OR REPLACE PROCEDURE procedure_name (parameters)
IS
BEGIN
    -- SQL statements
END;
```

EXAMPLE:

```
CREATE OR REPLACE PROCEDURE GetStudent(p_id IN NUMBER)
IS
    v_name VARCHAR2(50);
BEGIN
    SELECT Name INTO v_name
    FROM Students
    WHERE StudentID = p_id;

    DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_name);
END;
```

```
EXEC GetStudent(1);
```

PL/SQL Functions

A PL/SQL function is a named block that performs a specific task and always returns a single value using the `RETURN` statement. It can be used in SQL statements as well as in PL/SQL blocks.

SYNTAX:

```
CREATE OR REPLACE FUNCTION function_name (parameters)
RETURN datatype
IS
BEGIN
    -- statements
    RETURN value;
END;
```

EXAMPLE:

```
CREATE OR REPLACE FUNCTION get_grade(marks NUMBER)
RETURN VARCHAR2
IS
    grade VARCHAR2(2);
BEGIN
    IF marks >= 90 THEN
        grade := 'A';
    ELSIF marks >= 75 THEN
        grade := 'B';
    ELSIF marks >= 60 THEN
        grade := 'C';
    ELSE
        grade := 'D';
    END IF;

    RETURN grade;
END;
```

```
SELECT get_grade(85) FROM dual;
```

Procedure	Function
A procedure does not return a value directly.	A function must return a single value.
The RETURN statement is not mandatory.	The RETURN statement is mandatory.
It is used to perform actions such as insert, update, or delete operations.	It is used to perform calculations and return a result.
It is called as a standalone statement.	It can be called inside expressions or SQL queries.
It can use IN, OUT, and IN OUT parameters.	It mainly uses IN parameters.
It cannot be used in SELECT statements.	It can be used in SELECT statements.
Its main purpose is to execute a set of operations.	Its main purpose is to compute and return a value.

Triggers

Triggers are a feature in MySQL that allow you to **execute(or fire) a set of actions automatically in response to specific events**, such as the insertion, update, or deletion of data in a table.

Structure of a Trigger:

- **Trigger Definition:** Specifies when the trigger should fire (event and timing).
- **Trigger Body:** Contains the SQL statements to be executed.

SYNTAX:

```
CREATE TRIGGER trigger_name
BEFORE | AFTER INSERT OR UPDATE OR DELETE
ON table_name
FOR EACH ROW
BEGIN
    -- SQL statements
END;
```

EXAMPLE:

```
CREATE TRIGGER update_last_update
AFTER UPDATE ON mytable
FOR EACH ROW
BEGIN
    SET NEW.last_update = NOW();
END;
```

This trigger runs automatically after an update on `mytable` and sets the `last_update` column to the current time for each updated row.

Types of Triggers:

- BEFORE Trigger – Executes before the event
- AFTER Trigger – Executes after the event

Advantages:

1. Maintains data integrity
2. Automates repetitive tasks
3. Helps in auditing and logging changes
4. Enforces business rules automatically

Disadvantage:

Triggers can be complex and difficult to debug if not used carefully.

Assertions

An assertion is a condition (predicate) defined on a database that must always remain true.

Key Points:

- It enforces a rule on the entire database.
- The condition must be true when the assertion is created.
- The DBMS checks the assertion after every insert, update, or delete operation.

SYNTAX:

```
CREATE ASSERTION assertion_name
CHECK (condition);
```

EXAMPLE:

```
CREATE ASSERTION city_not_null
CHECK (
  NOT EXISTS (
    SELECT *
    FROM customer
    WHERE customer_city IS NULL
  )
);
```

This assertion ensures that no record in the `customer` table has a NULL value in the `customer_city` column. If any row violates this condition, the operation will be rejected.

Assertion	Trigger
Assertions do not modify data; they only check conditions.	Triggers are more powerful because they can check conditions and also modify data.
Assertions are not linked to specific tables or events.	Triggers are linked to specific tables and events (INSERT, UPDATE, DELETE).
All assertions can be implemented using triggers.	Not all triggers can be implemented as assertions.
Assertions are not supported in Oracle.	Triggers are supported in Oracle.

Roles and privileges

A role is a collection of privileges that can be assigned to users to simplify access control and database administration.

Example of roles:

```
CREATE ROLE data_manager;

GRANT CREATE TABLE, CREATE VIEW TO data_manager;
```

Privileges are permissions given to users to perform specific operations on the database.

Type	Description	Example
System Privilege	Allows user to perform actions like CREATE, ALTER, DROP	CREATE TABLE , CREATE VIEW
Object Privilege	Allows user to perform operations on data	SELECT , INSERT , UPDATE , DELETE

Example of privilege:

```
GRANT SELECT, INSERT ON Students TO user1;
```